コマンドラインの使い方

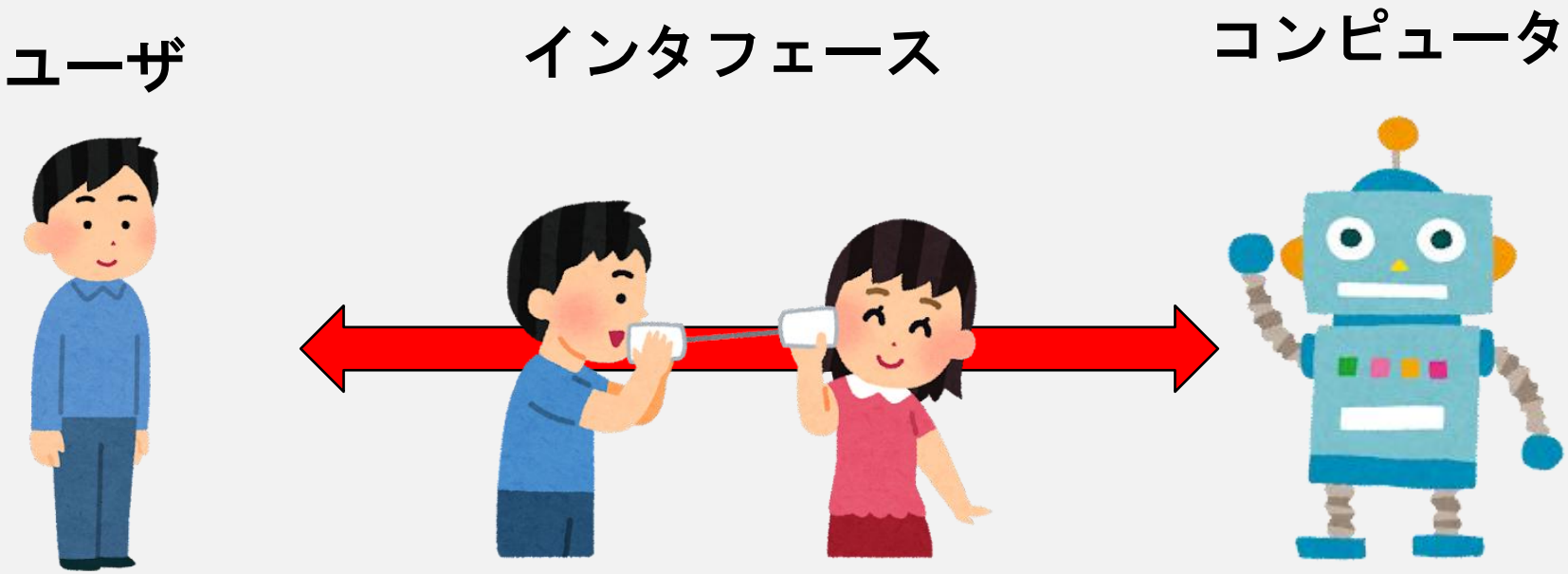
慶應義塾大学理工学部物理情報工学科
渡辺

はじめに

Gitは、もともとLinuxの開発のために作られた
コマンドラインから操作することが前提
本講義でもGitを主にコマンドラインから操作する

➡ そもそもコマンドラインとは何か？

インタフェースとは



コンピュータに何かさせるにはユーザが「指示」をする必要がある
指示を伝える手段が「インタフェース」

インタフェース

グラフィカルユーザインタフェース
(Graphical User Interface, GUI)

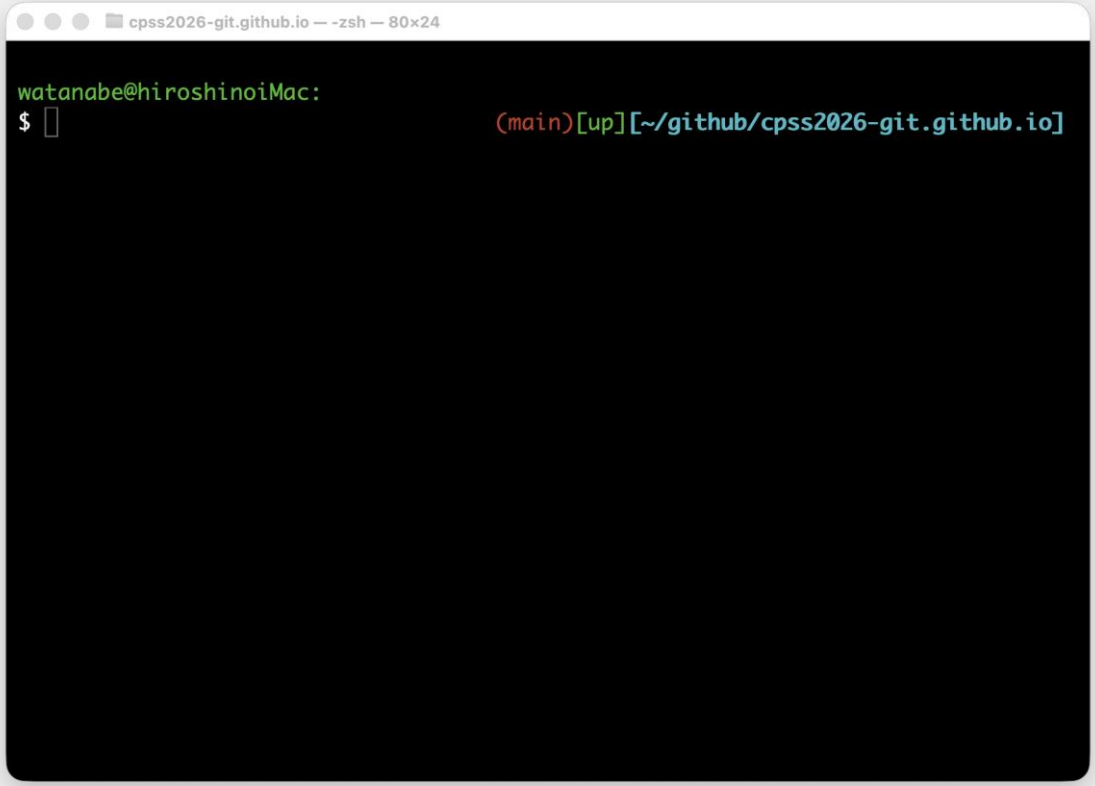
コンピュータへ出す指示を画面上で視覚的に表現する
操作や対象を抽象化したアイコンを利用する
マウスやタッチパネルなどを入力デバイスとする

コマンドラインインタフェース
(Command-Line Interface, CLI)

コマンド文字列でコンピュータへ指示を出す
キャラクタユーザインタフェース(CUI)とも
キーボードを主な入力デバイスとする
ターミナルから入力する

ターミナル

コマンドラインインターフェースの舞台は
WindowsならGit BashやWSL (Ubuntu)
MacやLinuxならターミナル

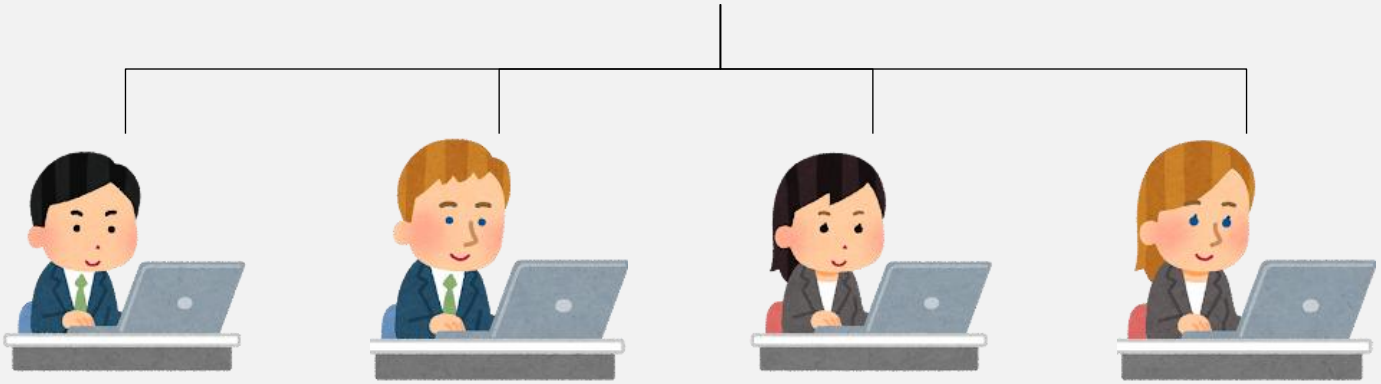


いわゆる「黒い画面」

ターミナル(端末)

もともとは大型コンピュータ(ホスト)に接続され、利用者が命令を送るための
インタフェースのこと

ホストコンピュータ



ターミナル(ユーザーインタフェース)

ホストコンピュータは高価であり、複数人で共有して利用するために
端末が必要だった

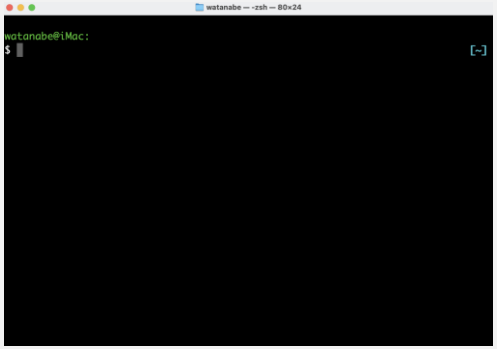
ターミナルエミュレータ

- 昔はタイプライターが端末であった(テレタイプ端末, TTYの語源)
- その後ディスプレイを使ったビデオターミナルが出現
- ホストコンピュータにはビデオターミナルが付属していたが、DECのVT100がターミナルのスタンダードに

ホスト		ターミナル
IBM System/360	←→	IBM 2260
DEC PDP-11	←→	VT100

VT100

ClickRick – CC-BY-SA 3.0
https://commons.wikimedia.org/w/index.php?curid=6693684



Macの「ターミナル」やWSLのUbuntuの画面は、「ターミナルエミュレータ」と呼ばれ、VT100の動作をエミュレートするものがほとんど

OS・シェル・ターミナル

オペレーティングシステム (Operating System, OS)

ハードウェアを抽象化し、資源を制御、割り当てるシステム

シェル (Shell)

ユーザのコマンド入力を解釈し、OS上でプログラムを実行する

ターミナル (Terminal)

入力と表示を提供するユーザインターフェース

コマンドプロンプト

コマンドプロンプト (command prompt)

シェルにおいて、ユーザからの入力待ちを示すもの
「$」で表現することが多い。

コマンド (command)

プロンプトに入力する命令
引数(ひきすう)やオプションを指定できる

```
$ ls -la dir
```

コマンドプロンプト　コマンド　オプション　引数

コマンドプロンプト

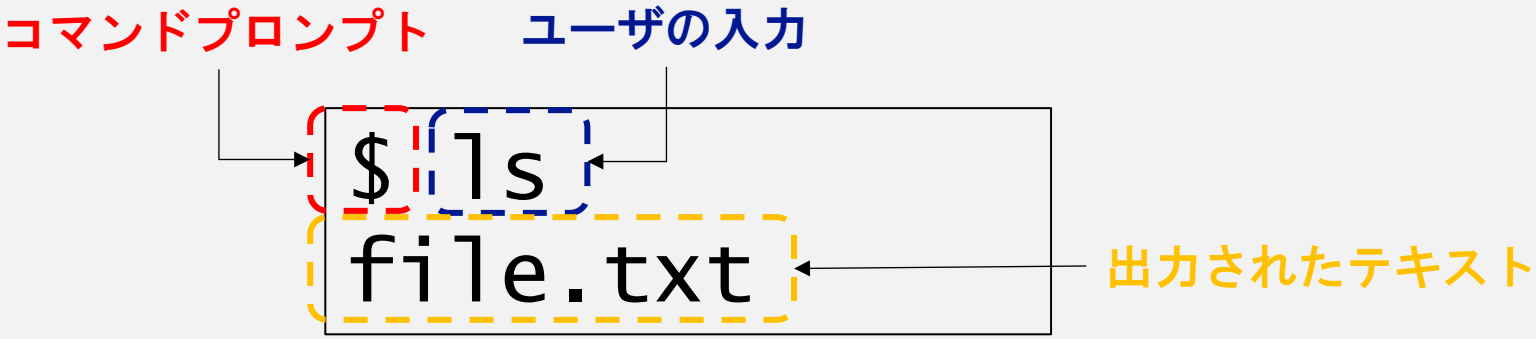
コマンドプロンプトは様々な表記がある。一行の場合もあるし、二行に渡る場合もある。右に情報を掲載する場合もある。

```
watanabe@MacAir ~ %
```

```
[watanabe@Server ~ ]$
```

```
watanabe@iMac:
$                                   [~]
```

以降はコマンドプロンプトを「$」で表記し、$で始まらないテキストはコマンドの出力であると約束する



ディレクトリとパス

ディレクトリ (Directory)

複数のファイルをまとめるフォルダのこと
以後、フォルダのことを「ディレクトリ」と呼ぶ

カレントディレクトリ (Current Directory)

ワーキングディレクトリとも
「現在自分が見ているディレクトリ」のこと
CUIだとどこにいるかわからなくなる場合があるので注意

ホームディレクトリ (Home Directory)

ターミナルを開いた直後のカレントディレクトリ
引数無しのcdコマンドで戻ってくる場所

ディレクトリとパス

パス (path)

階層構造の中でファイルの位置を示す文字列
デフォルトでファイルを探しに行くディレクトリ

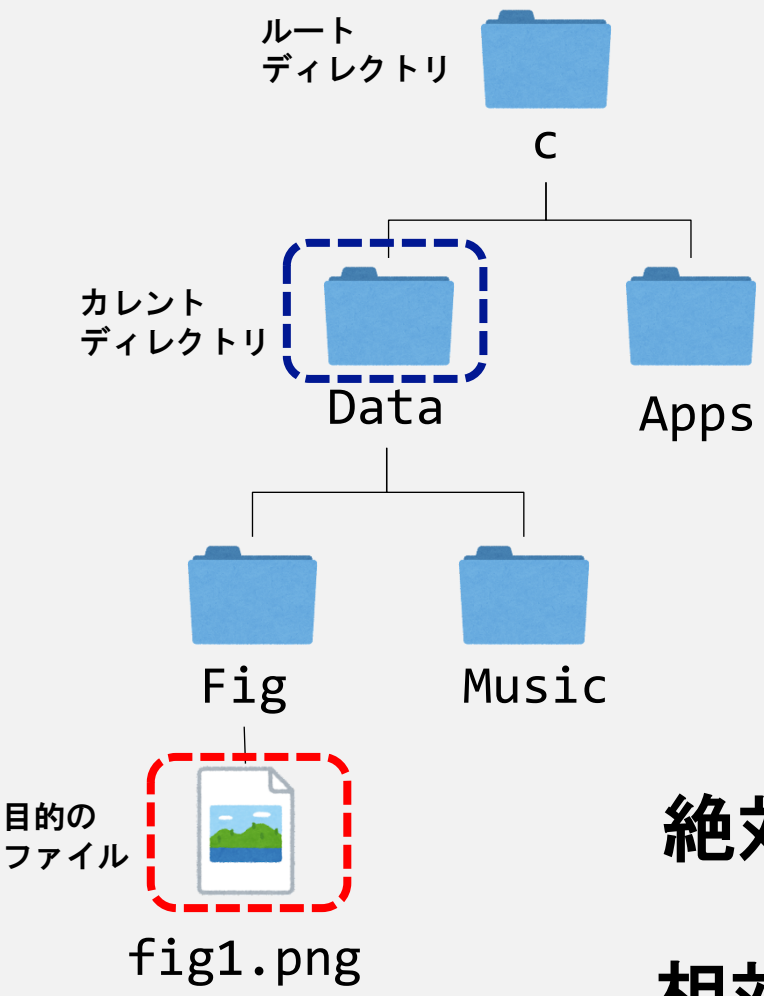
絶対パス (absolute path)

最上位階層から目的のファイルまでの位置
ルートディレクトリ

相対パス (relative path)

カレントディレクトリから目的のファイルまでの位置

ディレクトリとパス



絶対パス c/Data/Fig/fig1.png

相対パス Fig/fig1.png

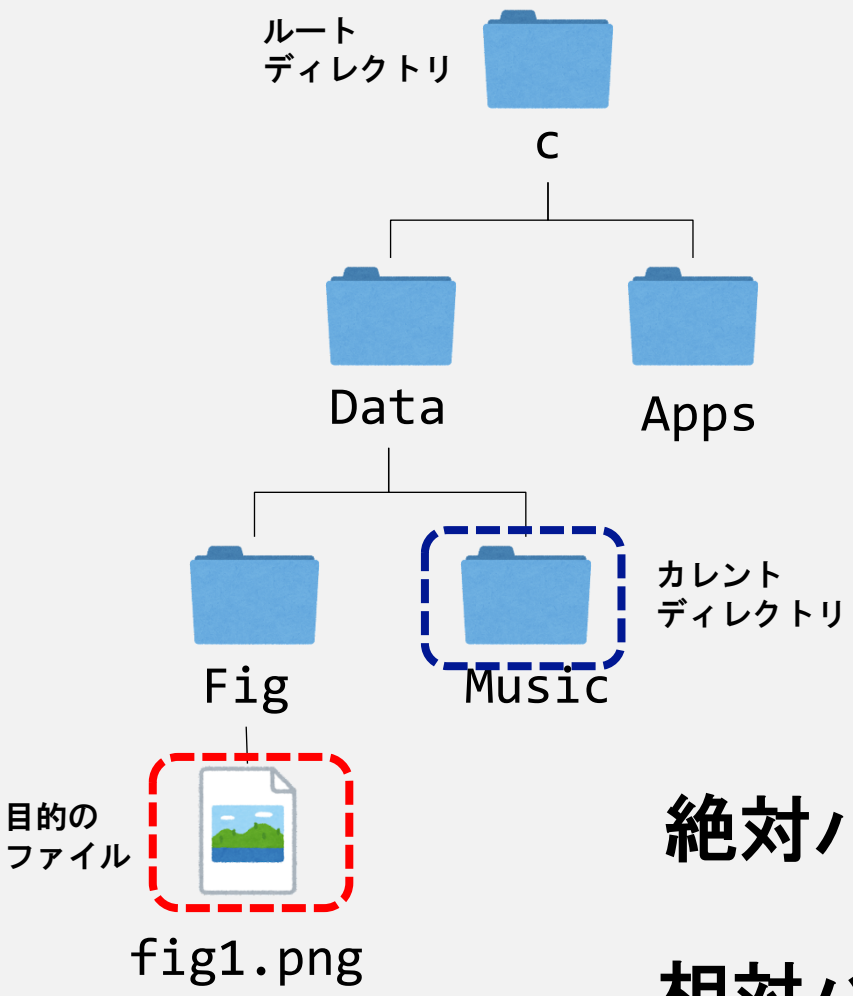
ディレクトリとパス

特別なディレクトリ

- . カレントディレクトリ
- .. カレントディレクトリの親ディレクトリ

相対パスで「真下」以外のパスを指定するのに使用

ディレクトリとパス



絶対パス c/Data/Fig/fig1.png

相対パス ../Fig/fig1.png

ls

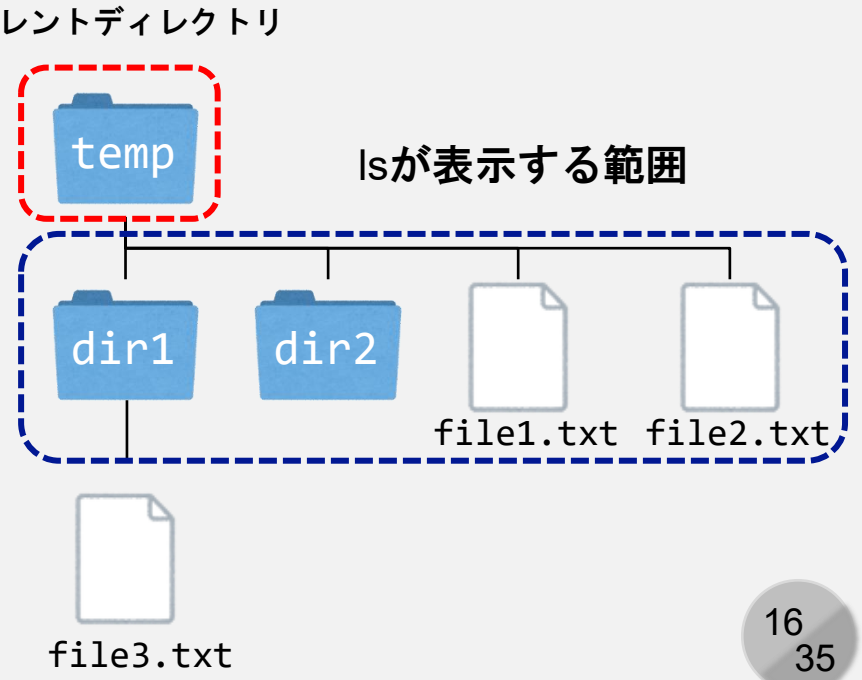
指定したディレクトリ内のファイルやディレクトリを表示する
引数を指定しない場合はカレントディレクトリ直下を表示する

引数 (argument)

コマンドの直接の目的語

```
$ ls
dir1/  dir2/  file1.txt  file2.txt
```

```
$ ls dir1
file3.txt
```



ls

オプション (option)

コマンドの振る舞いを変えるもの
ハイフンに続けて渡す
一度に複数指定することもできる

-l リスト表示

```
$ ls -l
total 0
drwxr-xr-x 1 watanabe 197121 0 Sep 26 23:06 dir1/
drwxr-xr-x 1 watanabe 197121 0 Sep 26 23:06 dir2/
-rw-r--r-- 1 watanabe 197121 0 Sep 26 23:06 file1.txt
-rw-r--r-- 1 watanabe 197121 0 Sep 26 23:06 file2.txt
```

-a 隠しファイルも表示する

```
$ ls -a
./  ../  dir1/  dir2/  file1.txt  file2.txt
```

エラーメッセージ

できない操作を指示するとエラーが表示される

例： lsの引数に存在しないディレクトリを指定した

```
$ ls non-existing-dir
ls: cannot access 'non-existing-dir': No such file or directory
```

エラーの現象と理由が表示される
現象：non-existing-dirにアクセスできない
理由：そんなファイルやディレクトリが存在しない

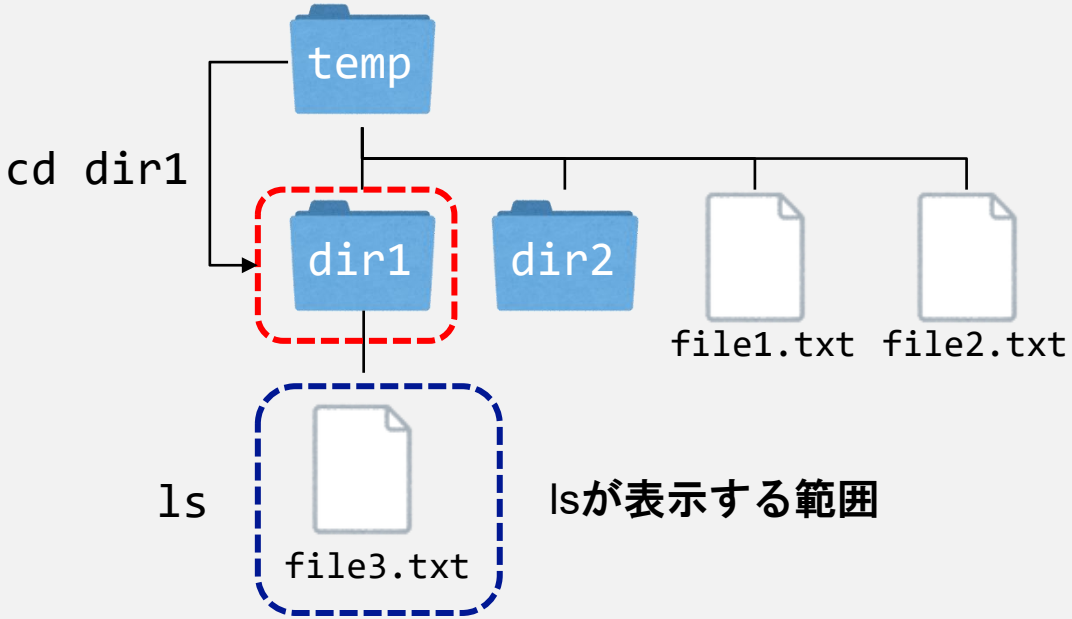
エラーメッセージは平易な英語なので読めるはず
ちゃんと読むのが脱初心者への近道
→ 読め！

cd

引数で指定したディレクトリをカレントディレクトリとする
引数を指定しない場合はホームディレクトリへ移動する

```
$ cd dir1

$ ls
file3.txt
```



cd

```
$ cd
```

引数無しではホームディレクトリへ移動する
場所がよくわからなくなったらこれを使う

```
$ cd ..
```

親ディレクトリへ移動する
よく使うので覚えておくこと

mkdir

引数で指定したディレクトリを作成する
(新規作成→フォルダ)
引数は必須

ディレクトリ dir3を作成

```
$ ls
dir1/  dir2/  file1.txt  file2.txt

$ mkdir dir3

$ ls
dir1/  dir2/  dir3/  file1.txt  file2.txt
```

ディレクトリ dir3が作成された

※ ディレクトリの削除はrmdirだが、あまり使わない

mv

mv 移動元 移動先

ファイルやディレクトリの移動、リネームを行う
「移動元」「移動先」と引数を二つとる
移動元は存在するファイルやディレクトリでなければならない
移動先が存在するかしないかで動作が変わる

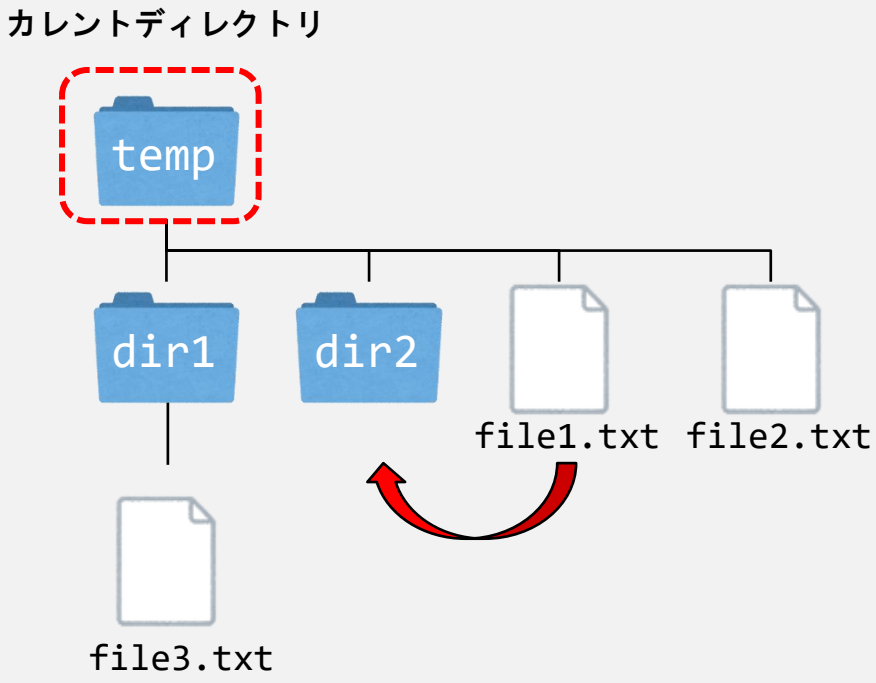
移動先が存在するディレクトリ
→そこに移動
移動先が存在しないファイルやディレクトリ
→移動元のファイルやディレクトリを移動した上でリネーム

mv

mv ファイル ディレクトリ

ファイルをそのディレクトリへ移動する

```
$ mv file1.txt dir2
```

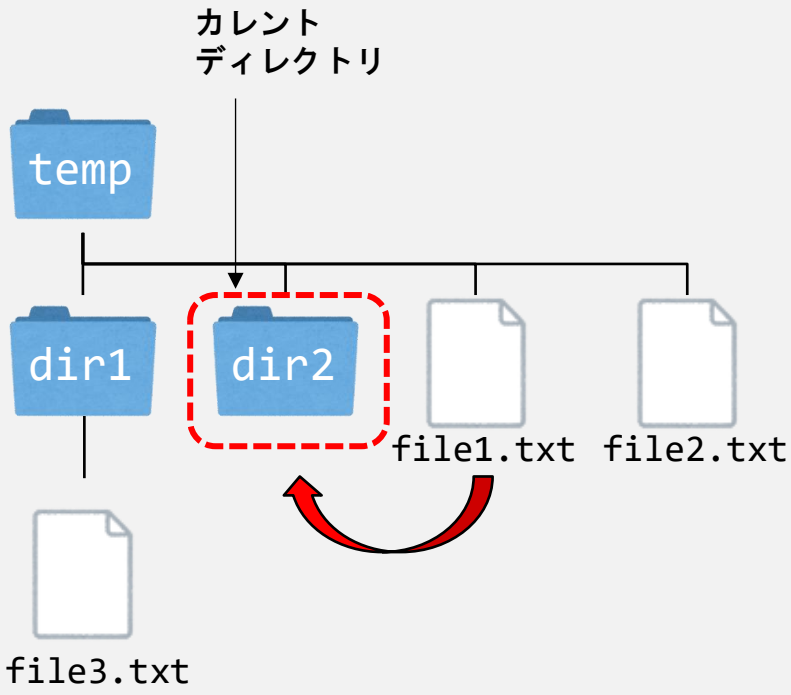


mv

mv ファイル ディレクトリ

ファイルをそのディレクトリへ移動する

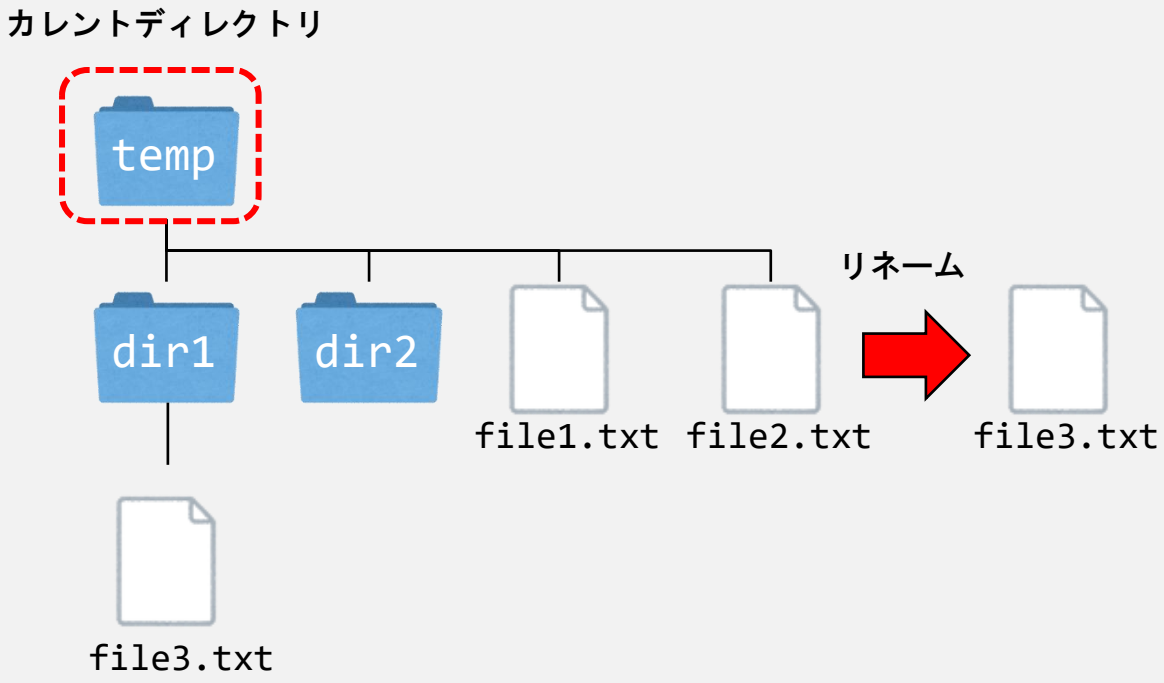
```
$ mv ../file1.txt .
```



mv

mv ファイル ファイル

```
$ mv file2.txt file3.txt
```

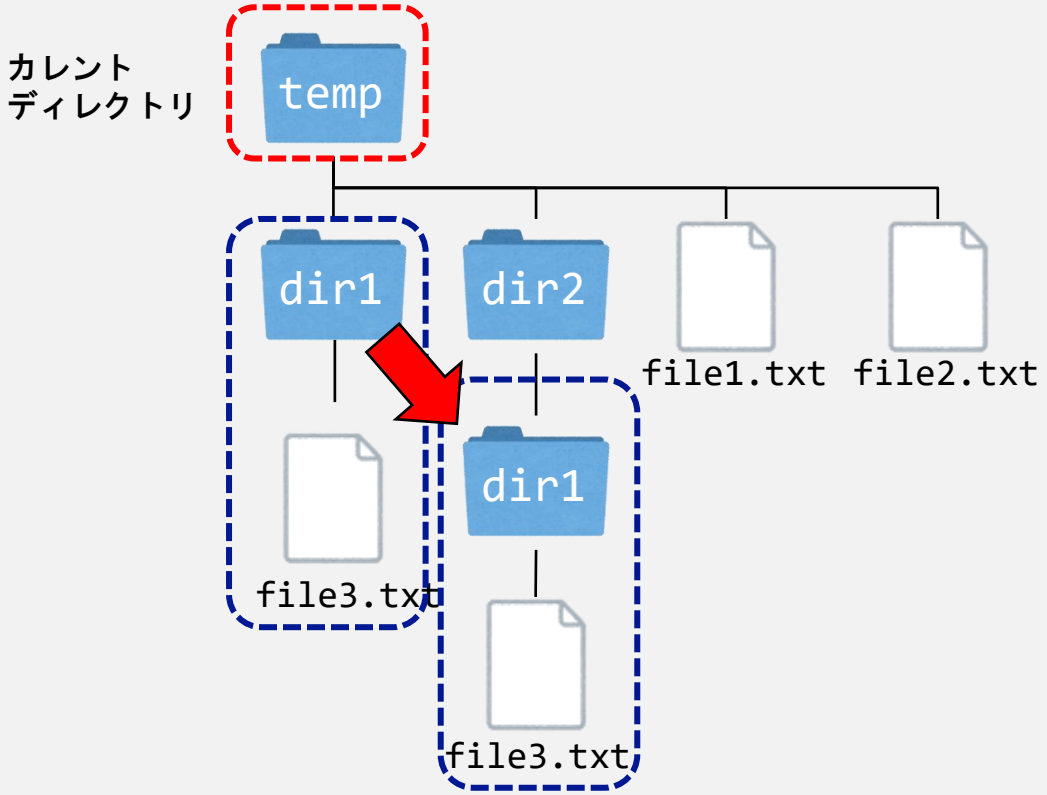


cp

ファイルをコピーする
移動元のファイルが消えない以外はほぼmvと同じ

```
$ cp -r dir1 dir2
```

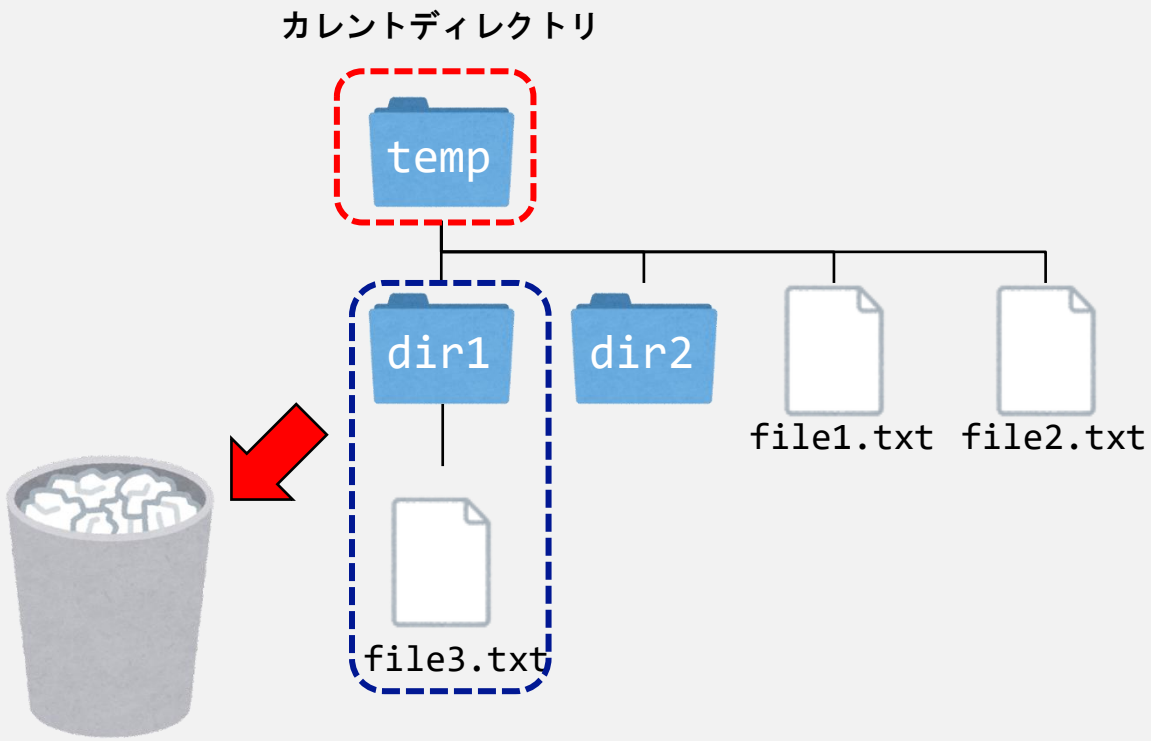
ディレクトリのコピーには-rオプションが必要



rm

ファイルやディレクトリを削除する
ディレクトリの削除には-rオプションが必要

```
$ rm -r dir1
```



※ 問答無用で中身ごと削除するので注意。-iオプションを付けると確認する

catとecho

cat ファイル名

ファイルの中身を表示する

echo メッセージ

メッセージを表示する

echo メッセージ > ファイル名

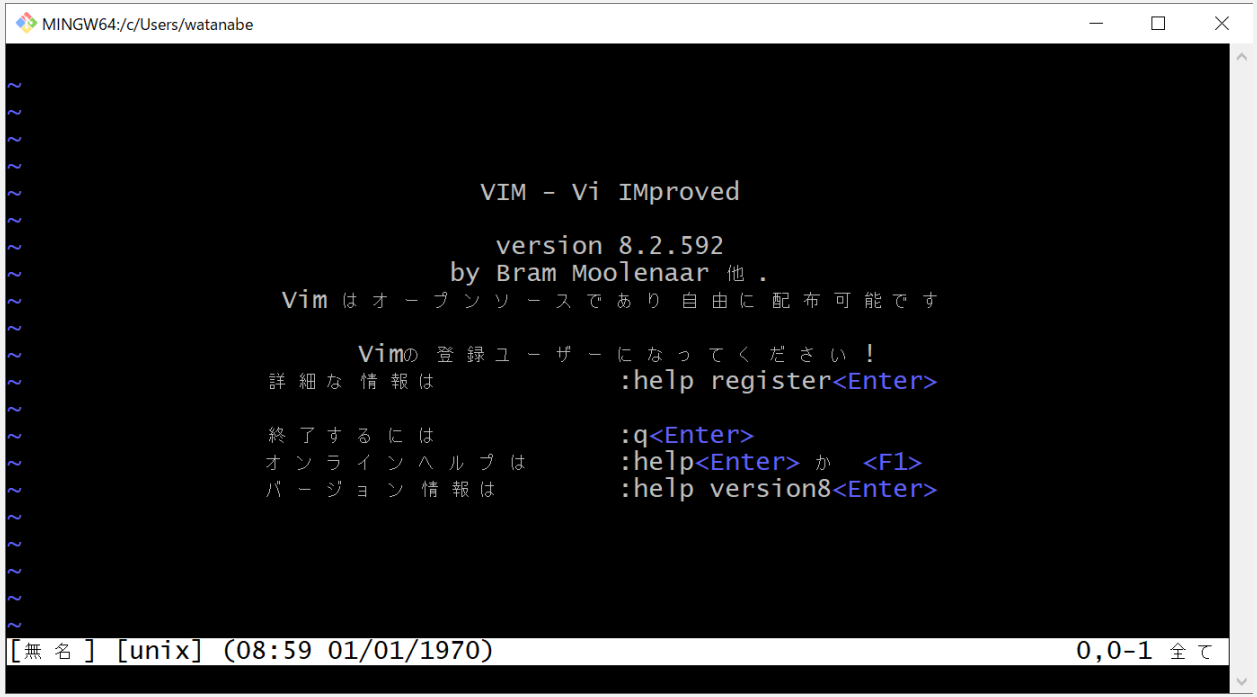
メッセージを中身とするファイルを作成

echo メッセージ | プログラム

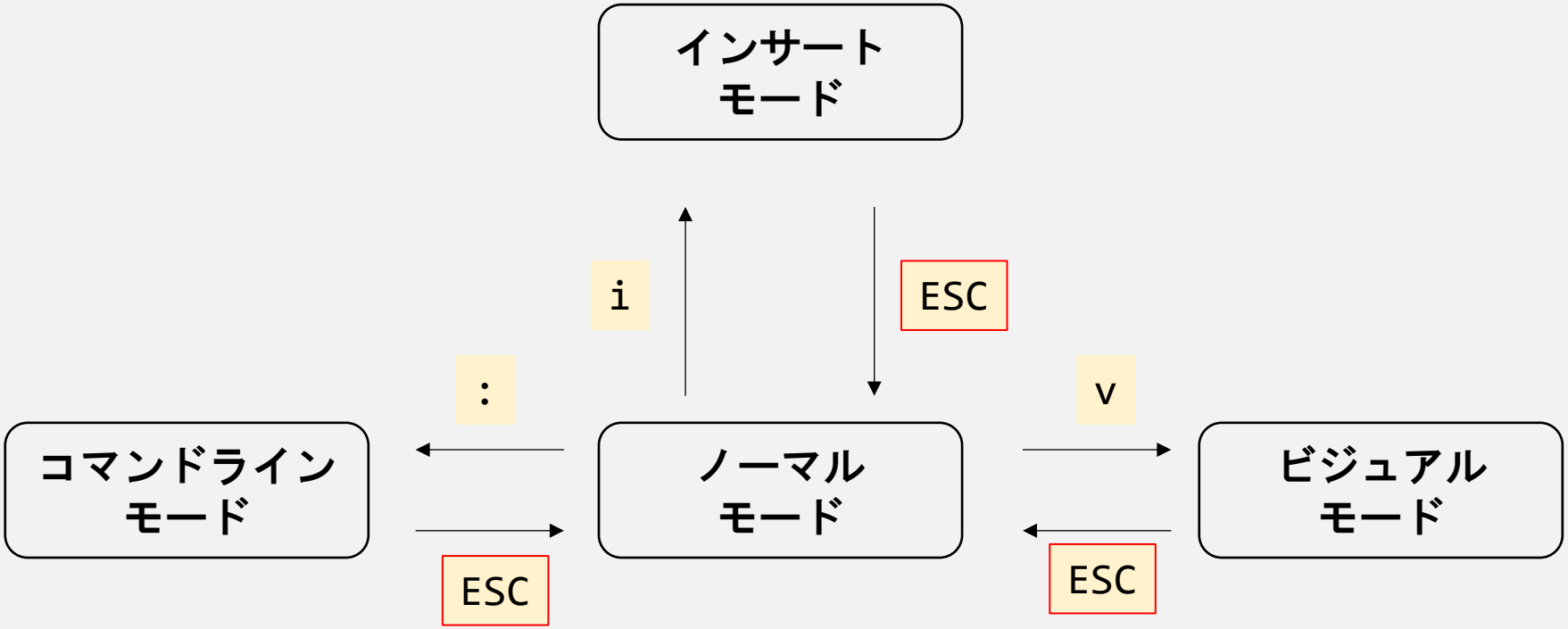
メッセージをプログラムの入力として実行

Vim

Vim (ヴィム)
多くの環境でGitのデフォルトエディタ
モードという概念がある
とっつきにくいが慣れると使いやすい

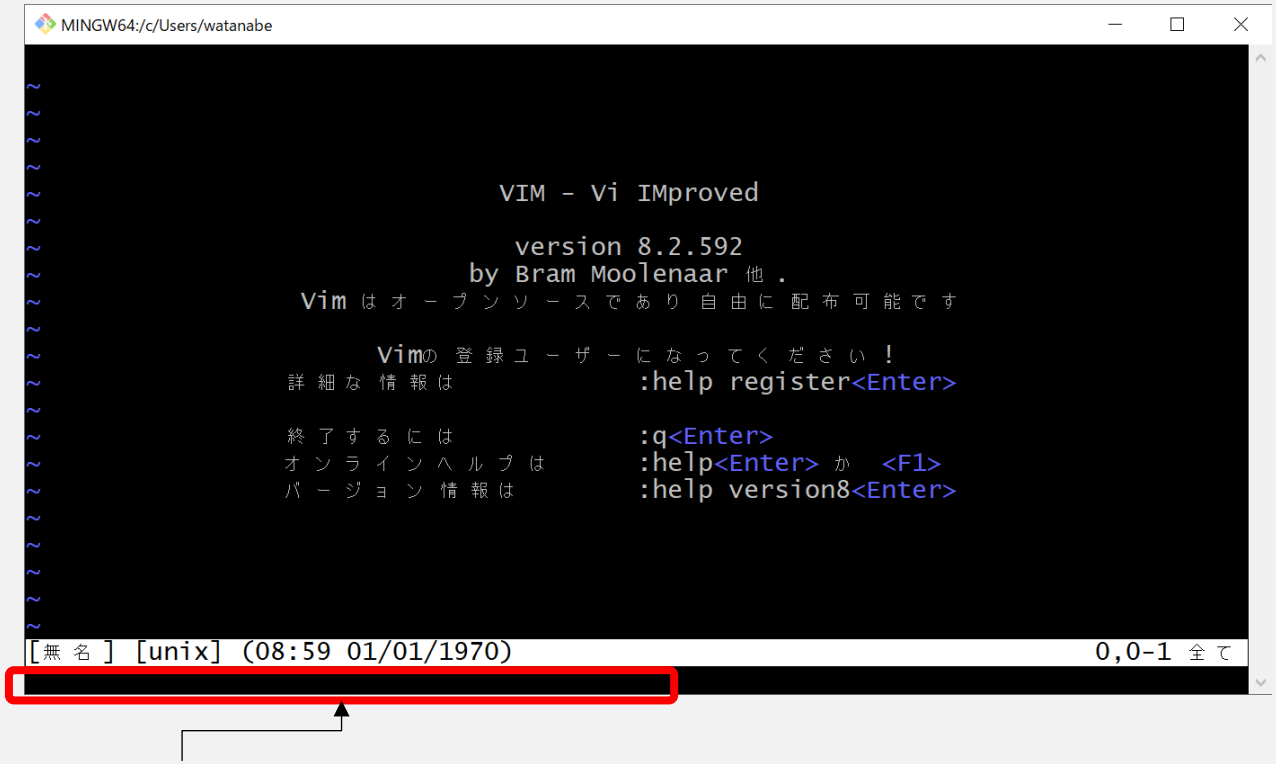


Vimのモード



ゴリラと学ぶVim講座 https://knowledge.sakura.ad.jp/21687/

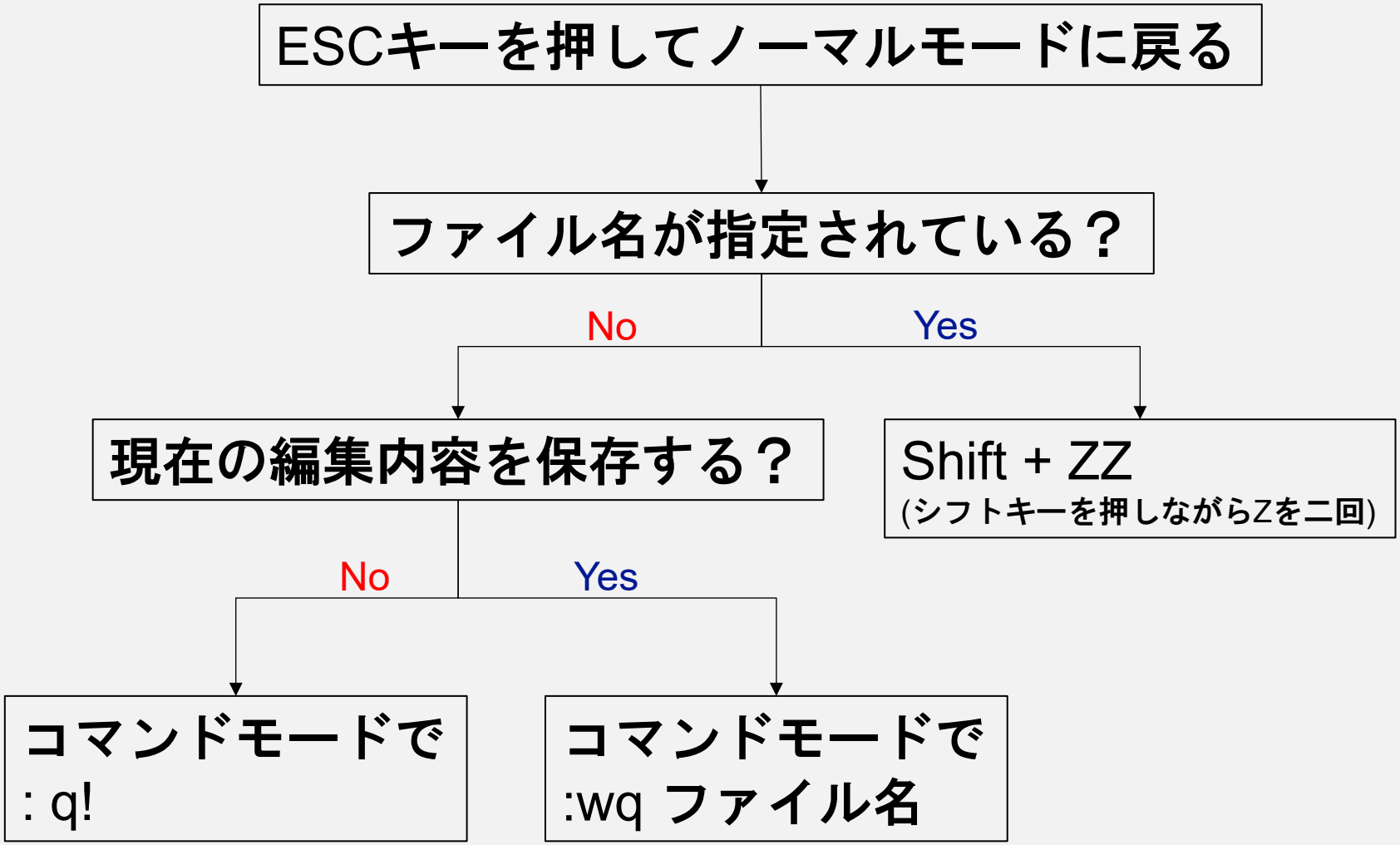
Vimのモード



Vimのモードはここを見ればわかる

ノーマルモード　：何も表示されない
インサートモード：「-- INSERT --」と表示
ビジュアルモード：「-- VISUAL --」と表示
コマンドモード　：「:」が表示され入力待ちに

Vimの終了の仕方



※ 既存のファイルを上書きする場合はwq!

VimとGit

GitからVimが呼び出される場合、ファイル名が指定された状態、かつノーマルモードで起動される

その後、以下の作業をすればよい

1. 「i」を押してインサートモードへ
2. 内容を編集
3. エスケープキーを押してノーマルモードへ
4. 「ZZ」でエディタを抜ける

Vimのノーマルモードでの編集

ノーマルモードで様々な編集ができる
とりあえずこれだけ覚えておく(後で使います)

`h` `j` `k` `l` カーソル移動

`dd` カーソル位置の行を切り取り

`p` 切り取った行をカーソル位置の行の下に貼り付け

`P` 切り取った行をカーソル位置の行の上に貼り付け

まとめ

インタフェースについて

インタフェースにはコマンドラインとグラフィカルなものがある
本講義ではGitを主にコマンドラインで使う
コマンドラインインタフェースでは、プロンプトにコマンドを入力する
コマンドには引数やオプションを与える
エラーメッセージを読む

Vim

本講義ではGitのエディタとしてVS Code及びVimを使う
Vimにはモードがある
困ったらエスケープキー
終了は「ZZ」